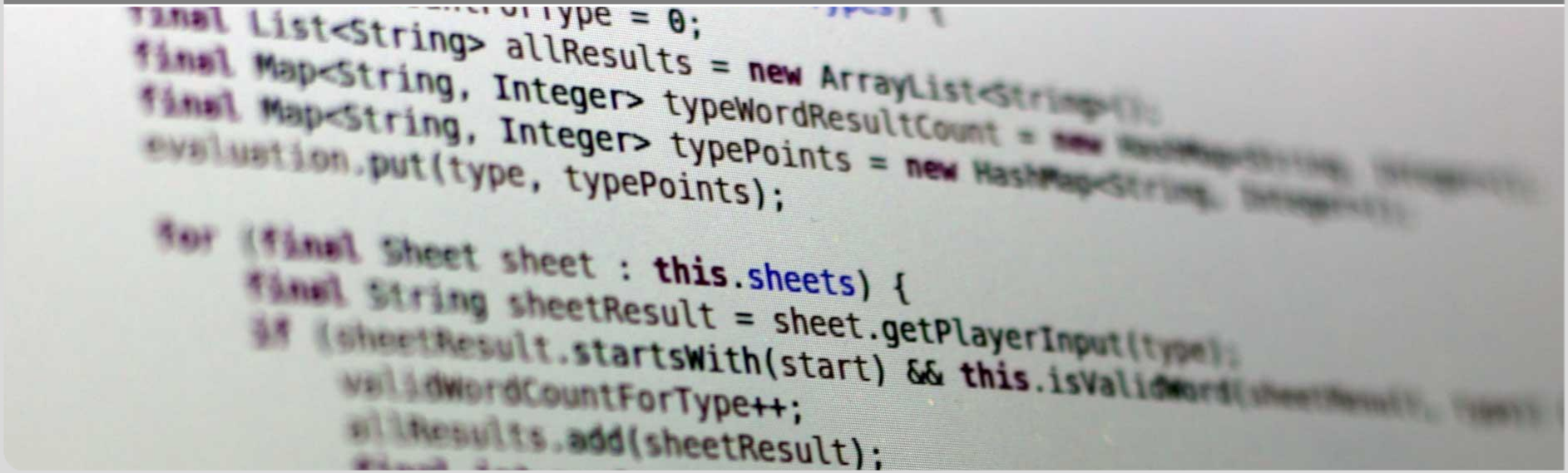
Karlsruher Institut für Technologie

DHBW
Duale Hochschule
Baden-Württemberg

Programmieren I+II
Informationen & Übung zur Prüfung

Download:
`https://www.iai.kit.edu/javavl/pe/uebung26.zip`

Institut für Automation und angewandte Informatik



Termin Programmentwurf

Montag, 12. Oktober 2026

09:00 – 12:00 Uhr

Kurs	Dozent	Raum
TINF25B1	W. Süß	A263
TINF25B2	T. Schlachter	A262
TINF25B4	J. Sidler	A265
TINF25B5	M. Koubaa	A267
TINF25B6	C. Schmitt	A266

Termin Programmentwurf

Montag, 12. Oktober 2026

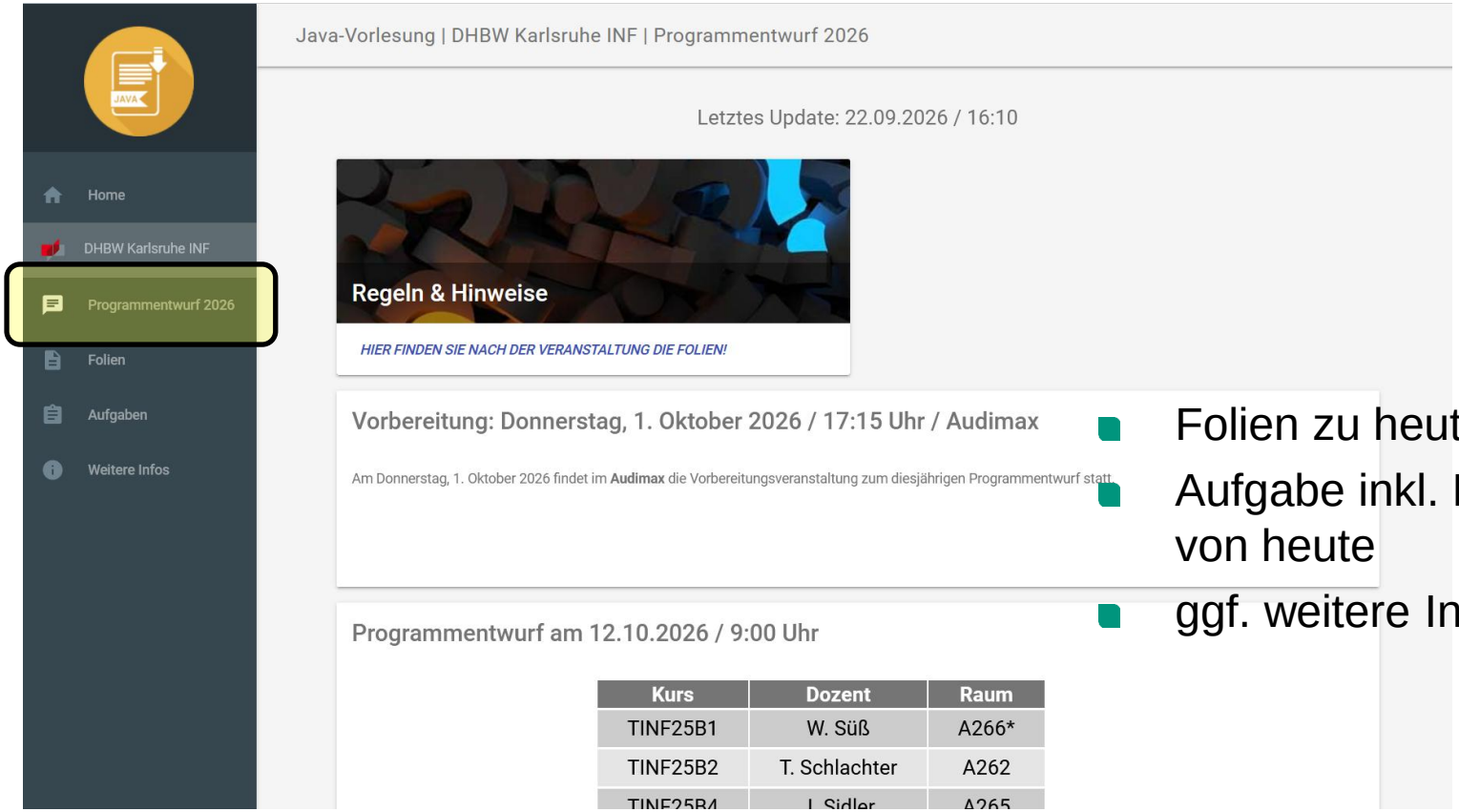
09:00 – 12:00 Uhr

Teil I (auf Papier)	Teil II (am eigenen Laptop)
Bearbeitungszeit: 30 Minuten Gewichtung in Gesamtnote: 25% Erlaubte Hilfsmittel: • **keine**	Bearbeitungszeit: 90 Minuten Gewichtung in Gesamtnote: 75% Erlaubte Hilfsmittel: • Vorlesungsfolien • Übungsaufgaben • Bücher zu Java-Programmierung Nicht erlaubt: • lokalen LLMs / andere KIs! • Netzzugang / Verbindung zu anderen Rechnern (auch BT*)

**gilt auch für Bluetooth-Mäuse!*

Weitere Informationen zur Prüfung

- Infos zum Programmentwurf auf der Homepage



- Folien zu heute
- Aufgabe inkl. Lösung von heute
- ggf. weitere Infos

https://www.iai.kit.edu/javavl/

Relevante Inhalte

- Stoff 1. Semester ✓
  - Primitive Datentypen, Kontrollstrukturen, Arrays
  - Klassen, Vererbung, Strings, Wrapperklassen, Pakete
  - Exceptions, I/O
- Interfaces, innere und anonyme Klassen ✓
- GUI mit Swing inkl. Eventbehandlung ✓
- Zeichnen ✗
- Datenstrukturen (Collection-API), enum ✓
- Threads ✓
- Java 8+-Features (erlaubt, aber nicht gefordert, JDK8+ dennoch Pflicht!) ✓
- Maven ✗
- XML, GSON ✗
- JUnit ✗

Allgemeines - Die Basics

- Aus organisatorischen Gründen werden Ihnen Sitzplätze zugewiesen



(Symbolbild)

- Studierendenausweis mitbringen
- Wir beginnen die Prüfung **pünktlich!**
- Zwischen Teil I und Teil II gibt es eine *kurze* Pause (~5min)

Symbolbild erzeugt mit Adobe Firefly / GPT Image 2.5 Sunburst

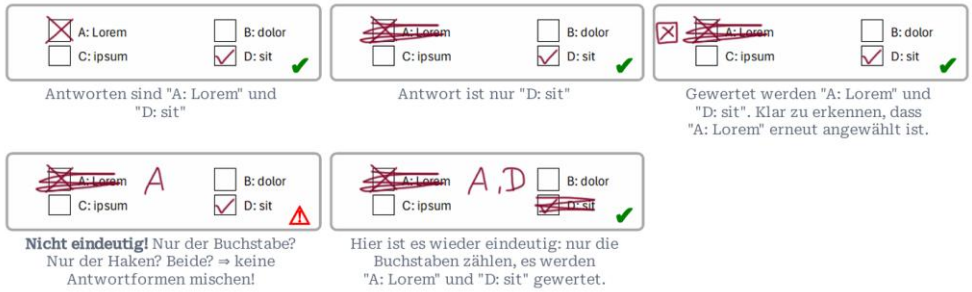
Allgemeines - Teil I

- Keine langen Prosa-Texte als Antwort, sondern
  - Multiple Choice
  - Lückentexte
  - *Kurze* Freitext-Antworten
  - Code-Snippets mit zu findenden Fehlern

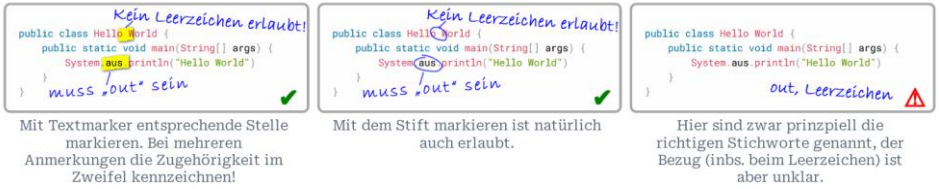
Antipattern siehe Folien „Zum Programmentwurf“ (F7-12)
https://www.iai.kit.edu/javavl/data/dhbw/2025-26/35_ZumProgrammentwurfUpdate2026.pdf

- Antworten eindeutig kennzeichnen!

Beispiel: Ankreuzen



Beispiel: Markieren/Anmerken



Allgemeines - Teil II

- **Java-Version:** *mindestens* 1.8 (erlaubt: bis 27)

- **Check your Laptop!**
  - Laptop + IDE rechtzeitig auf Funktionsfähigkeit prüfen!
  - Wenn Betriebssystem und/oder Entwicklungsumgebung nicht funktionieren kann darauf keine Rücksicht genommen werden!
    ➔ **„Reparatur“ geht von der Bearbeitungszeit ab!**

- **Abgabe**: Java-Quellcode auf USB-Stick
  - Falls fremde USB-Sticks aufgrund von Restriktionen des Firmenlaptops ein Problem sind vorab Bescheid geben!
  - Verwendete Bibliotheken, die nicht zum Java SE-Standard gehören, müssen der Lösung beigelegt werden!



Allgemeines - Teil II

- Nochmal zum „Reizthema“ KI
  - Unsere Vorgabe: **Keine KI während der Prüfung** erlaubt
  - Uns ist bewusst: Abgrenzung teils schwierig
  - Gute Daumenregel: „Deterministische Code-Completion“: passt
  - Vorbereiteter Code für 1:1 Copy&Paste + selbst anpassen: passt

- Wenn Sie Dinge vorab mit KI-Hilfe aufbereiten und das während der Prüfung *ohne* die KI nutzen: passt

- Aber: allerspätestens wenn es wie auf der nächsten Folie aussieht, ist es definitiv nicht erlaubt
  ➔ Überprüfbarkeit während der Prüfung durch Aufsicht

Personalisiert

Temporärer Chat

Dieser Chat kann Plugins und benutzerdefinierte Anweisungen berücksichtigen, erscheint aber nicht in deinem Verlauf.

Schreibe mir bitte ein Java-Programm, welches mir “Hello World” in 10 verschiedenen Sprachen (also Sprachen wie deutsch, englisch, ...) zeigt, je nachdem welches Locale die JVM hat.

+ Nachdenken

Aus Sicherheitsgründen können wir bis zu 30 Tage lang eine Kopie des Chats aufbewahren.

Allgemeines - Teil II

- Aber auch ohne die Animation wie bei Chatbots

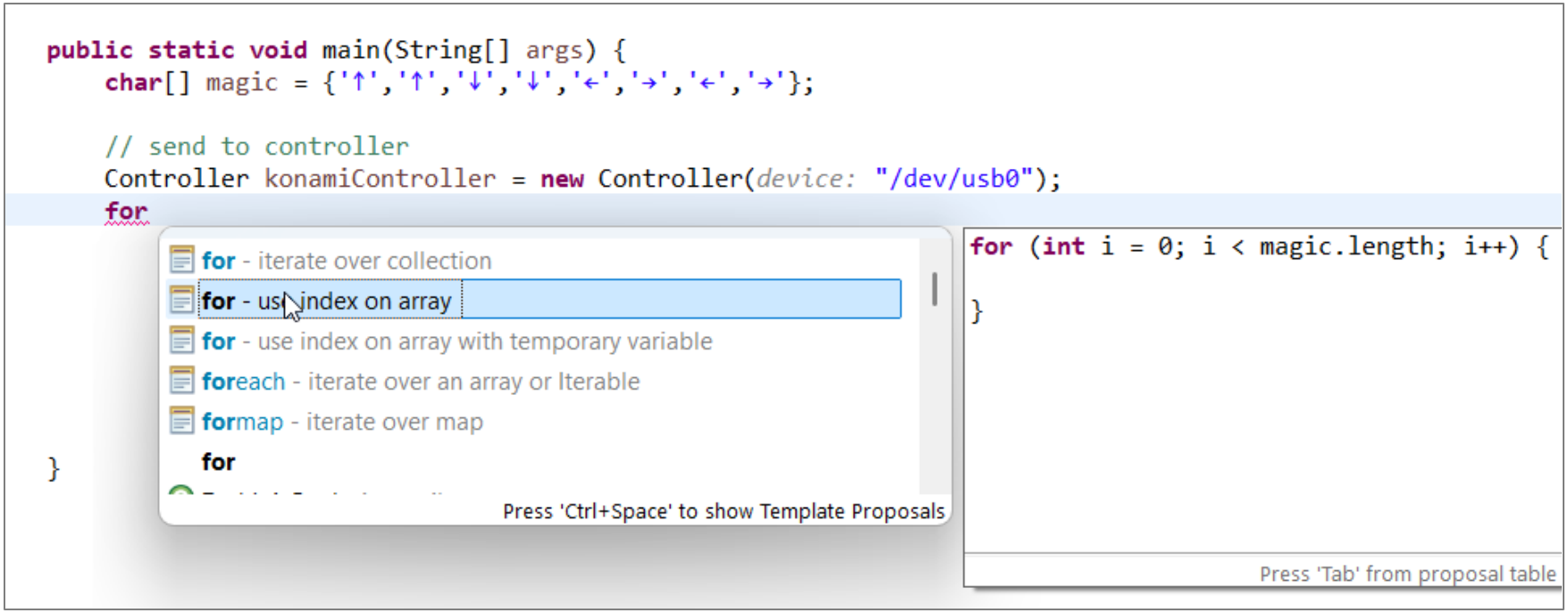
```
public static void main(String[] args) {
    char[] magic = {'↑','↑','↓','↓','←','→','←','→'};

    // send to controller
    Controller konamiController = new Controller(device: "/dev/usb0");
    for

}
```

Allgemeines - Teil II

- Aber auch ohne die Animation wie bei Chatbots



Allgemeines - Teil II

- Aber auch ohne die Animation wie bei Chatbots

```
public static void main(String[] args) {
    char[] magic = {'↑','↑','↓','↓','←','→','←','→'};

    // send to controller
    Controller konamiController = new Controller(device: "/dev/usb0");
    for (int i = 0; i < magic.length; i++) {

    }



}
```

➔ **Normales Templating, natürlich völlig in Ordnung!**

Allgemeines - Teil II

- Aber auch ohne die Animation wie bei Chatbots

```
public static void main(String[] args) {
    char[] magic = {'↑','↑','↓','↓','←','→','←','→'};

    // send to controller
    Controller konamiController = new Controller(device: "/dev/usb0");
    for
}
```

Hier könnte Ihr Tool stehen!

Press 'Ctrl+Space' to show Template Proposals

(oder irgendein anderer Weg zum Aufrufen des Tools ;-)

Allgemeines - Teil II

- Aber auch ohne die Animation wie bei Chatbots

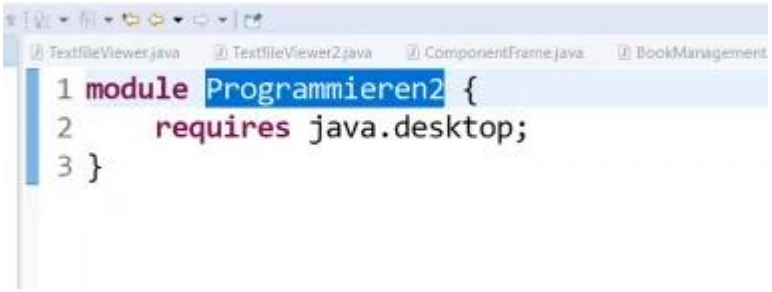
```
public static void main(String[] args) {
    char[] magic = {'↑','↑','↓','↓','←','→','←','→'};

    // send to controller
    Controller konamiController = new Controller(device: "/dev/usb0");
    for (int i = 0; i < magic.length; i++) {
        konamiController.sendButtonPress(buttonCode: magic[i]);
        try {
            // wait a little for remote device to process button
            Thread.sleep(millis: 10);
        } catch (Exception e) {
        }
    }
}
```

- Eindeutig inhaltlich erfasst „was zu tun ist“: passt nicht
- Im Zweifelsfall werden wir bei Ihnen nachfragen!

Was gibt es noch zu beachten?

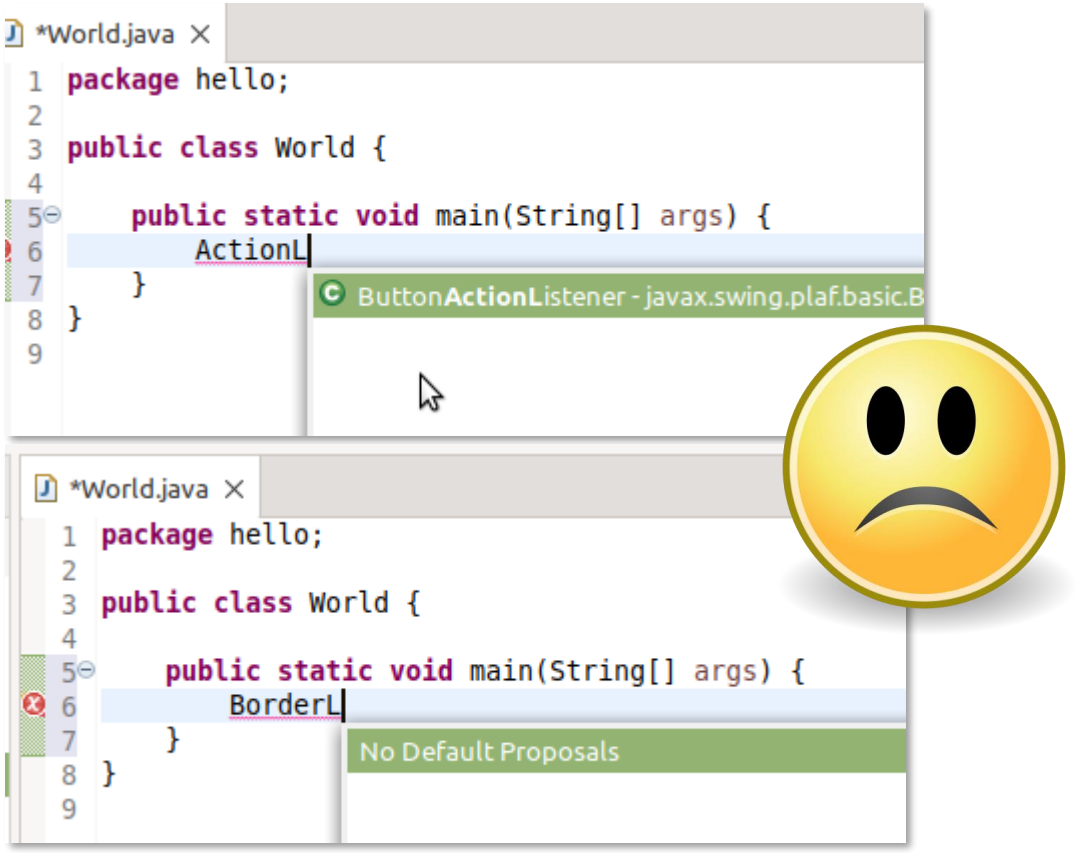
- Am besten neues Java-Projekt für die Prüfung anlegen
  - Im Idealfall vorab das Projekt schon anlegen
  - Projekt auch mit kleiner GUI-Testanwendung ausprobieren (Java 11+ benötigt evtl. die Aktivierung des Swing-Moduls!)



➔ **Ausführliche Erklärung siehe: https://youtu.be/c2A6xIA1g0E?t=22**

Neuere Eclipse-Versionen: AWT-Vorschläge (1)

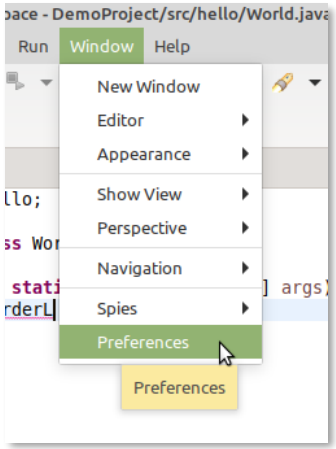
- Neuere Eclipse-Versionen schlagen standardmäßig keine `java.awt`-Klassen mehr vor (bspw: ActionListener)

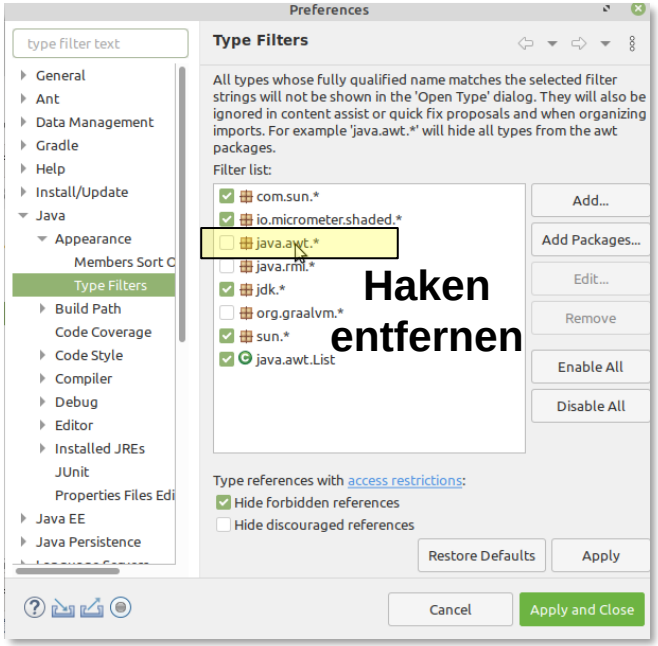


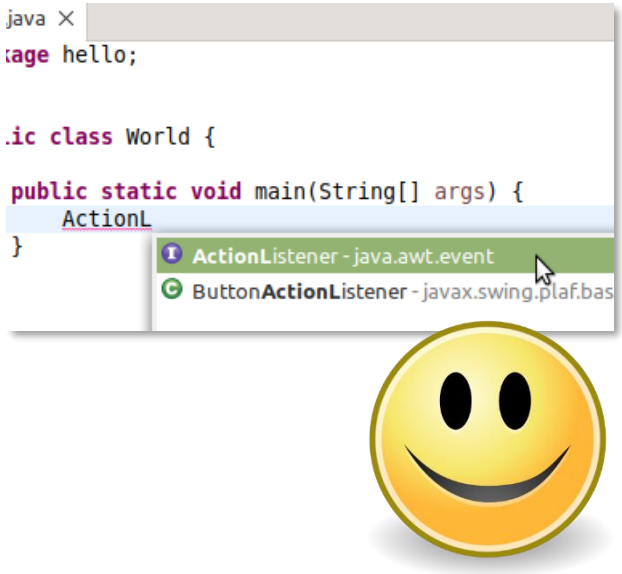
Emoticon by Tango / License: CC BY-NC 3.0 NL

Neuere Eclipse-Versionen: AWT-Vorschläge (2)

- Abhilfe: Type-Filter deaktivieren
  - Window → Preferences → Java → Appearance → Type Filters







Emoticon by Tango / License: CC BY-NC 3.0 NL

Zusammengefasst

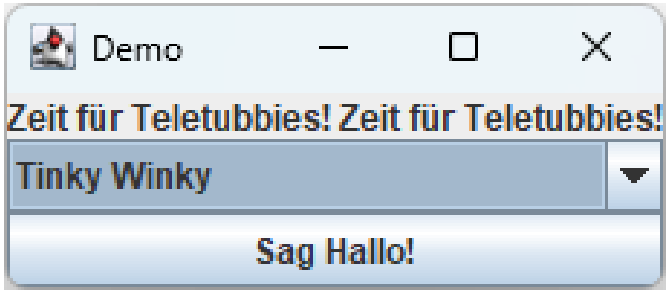
- **Zu checken**
  - „Prüfungsfit“?
  - Stifte dabei?
  - Laptop + IDE einsatzbereit?
  - Studierendenausweis dabei?
  - Ggf. Stromkabel (Verlängerung, Mehrfachstecker, Netzteil, …)?
  - Kann mein Laptop fremde Sticks? Wenn nein → melden!

- **Zu beachten**
  - Pünktlich sein!
  - Keine Netzverbindung bei der Prüfung
    → Materialien vorher auf Rechner kopieren!

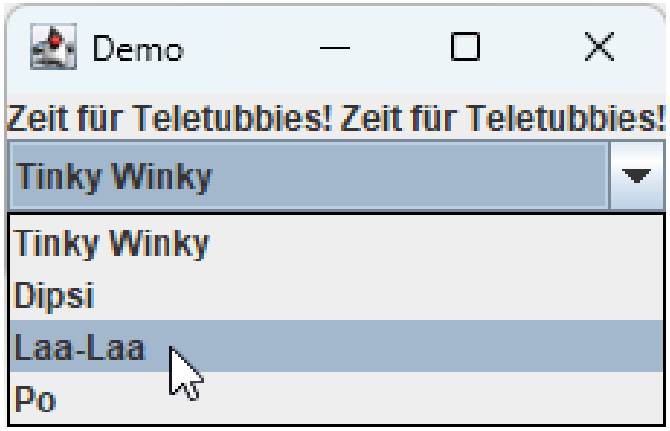
Aus „aktuellem Anlass“ (1)

- Manchmal vergessenes UI-Element: Auswahlbox

Ziel: UI zur Auswahl von „Personen“



Label
Auswahlbox
Button



4 Personen zur Auswahl



Button-Action, welche die Auswahl nutzt

Aus „aktuellem Anlass“ (2)

- Manchmal vergessenes UI-Element: Auswahlbox

Modellklasse in unserem Beispiel

```
public class Person {
    private String name;

    public Person(String name) {
        super();
        this.name = name;
    }
    public String getName() {
        return this.name;
    }
    /** Used for combo box! */
    @Override
    public String toString() {
        return this.name;
    }
}
```

Aus „aktuellem Anlass“ (3)

- Manchmal vergessenes UI-Element: Auswahlbox

```
public static void main(String[] args) {
   JFrame f = new JFrame("Demo");
   List<Person> persons = new LinkedList<>();
   persons.add(new Person("Tinky Winky"));
   persons.add(new Person("Dipsi"));
   persons.add(new Person("Laa-Laa"));
   persons.add(new Person("Po"));

   JComboBox<Person> cbxPersons = new JComboBox<>(persons.toArray(new Person[0]));
   JButton button = new JButton("Sag Hallo!");
   button.addActionListener(e->{
      Person selectedPerson = (Person)cbxPersons.getSelectedItem();
      // Person selectedPerson = persons.get(cbxPersons.getSelectedIndex()); // Alternativ
      JOptionPane.showMessageDialog(f, selectedPerson.getName() + " sagt: Ah Oh!");
   });

   f.add(new JLabel("Zeit für Teletubbies! Zeit für Teletubbies!"), BorderLayout.NORTH);
   f.add(cbxPersons, BorderLayout.CENTER);
   f.add(button, BorderLayout.SOUTH);
   f.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
   f.pack();
   f.setVisible(true);
}
```

Konstruktor nimmt nur Vector oder Array

getSelectedItem liefert nur Object → Typecast

Aus „aktuellem Anlass“ (4)

- Haben Sie eine Idee, was diese Zahl mit dieser Veranstaltung zu tun haben könnte?

1790867700

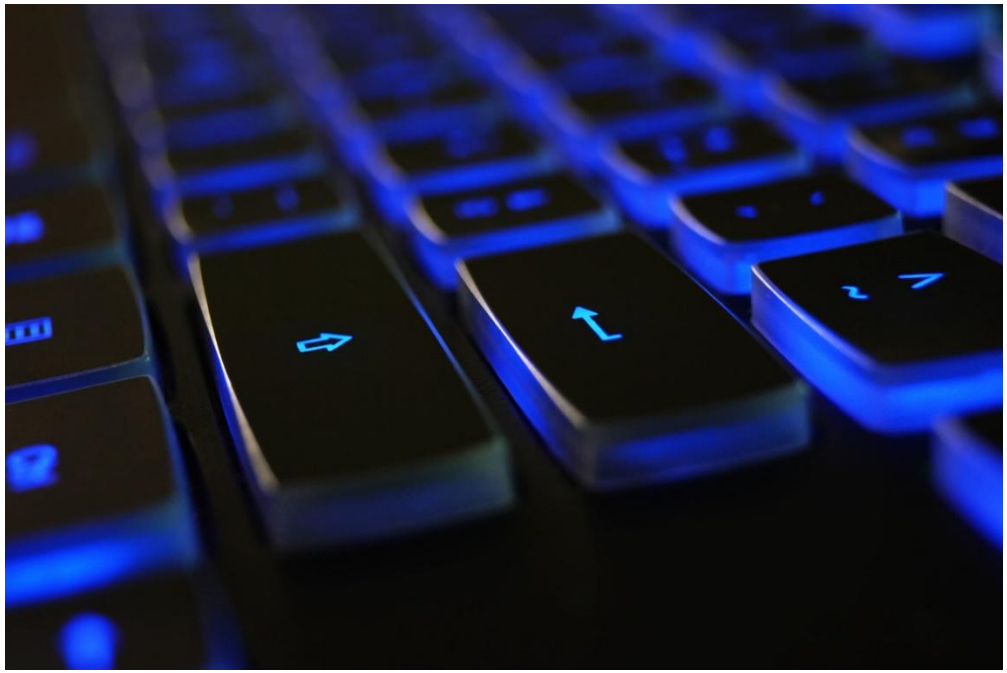
➔ **Sekunden seit dem 01.01.1970 für den geplanten Beginn dieser Veranstaltung**

Your Time Zone	Thu Oct 01 2026 17:15:00 GMT+0200 (Mitteleuropäische Sommerzeit)

- Das Kennen des Prinzips von Unix-Timestamps könnte beim Verständnis der Aufgabe helfen
  (das Konzept, nicht sowas wie formatierte Ausgabe)



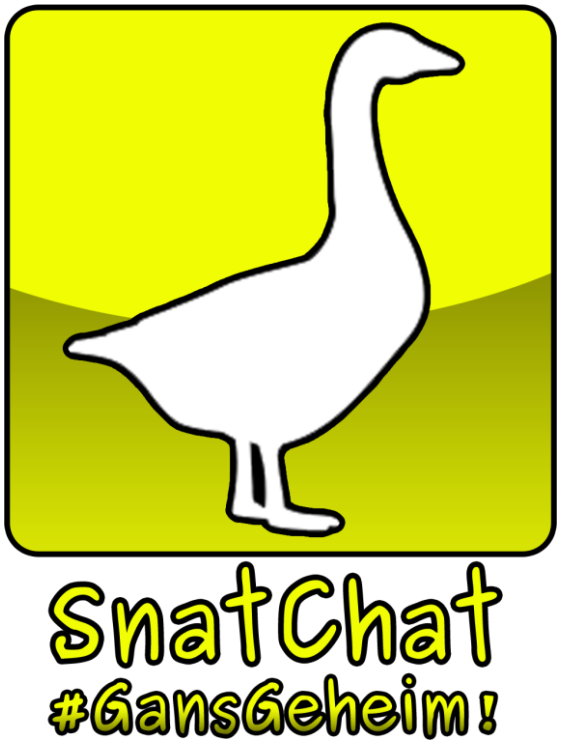
NOCH FRAGEN VON IHRER / EURER SEITE?



Nun zur

HEUTIGEN ÜBUNGSAUFGABE

Übungsaufgabe



https://www.iai.kit.edu/javavl/pe/uebung26.zip

(heute nur digital – bei der Prüfung dann auf Papier)

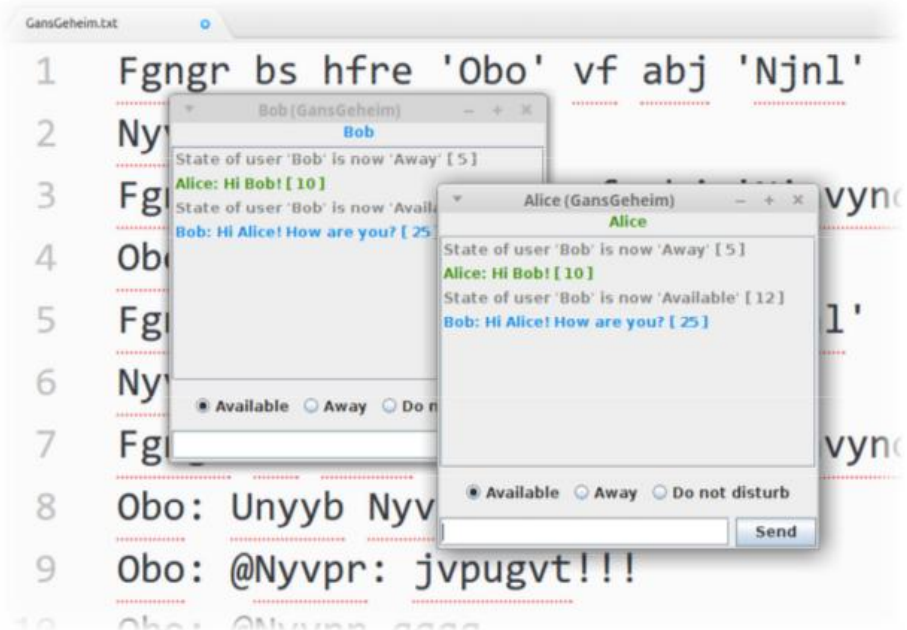
SnatChat – *#GansGeheim!*

Aufgabe

Schreiben Sie eine Java-Anwendung „*SnatChat*“, die einen Messenger für „geheime“ Kurznachrichten implementiert!

Nachrichten bei *SnatChat* zeichnen sich dadurch aus, dass sie für die Empfänger nur für kurze Zeit lesbar sind und sich danach – ähnlich wie in *Mission: Impossible* – „selbst zerstören“.

Der Nachrichtenaustausch ist über eine spezielle, Java-basierte Nutzeroberfläche „*SnatChatWindow*“ möglich. Im Hintergrund nutzen alle *SnatChatWindow*s zur Kommunikation einen *SnatChatRoom* um bspw. Nachrichten an alle Nutzer zu verteilen.



SnatChat – *#GansGeheim!*

Teilaufgabe a) **[5%]**

Durch den absehbaren, enormen Erfolg von *SnatChat* (Kurznachrichtendienste sind ein kaum besetztes Geschäftsfeld!) soll zunächst die Funktionalität für Frontends (wie *SnatChatWindow*) über das Java-*Interface* **`SnatChatFrontend`** definiert werden. Das Interface hat die folgenden Methoden:

- `public void receiveMessage(Message msg)`
- `public void receiveMessage(String text)`
- `public Account getAccount()`

SnatChat – *#GansGeheim!*

Teilaufgabe b) **[10%]**

Als Datenobjekte nutzt *SnatChat* Benutzerkonten (Klasse **`Account`**) sowie Nachrichten (Klasse **`Message`**).

Schreiben Sie zunächst einen *komplexen Aufzählungstyp* **`State`**, mit den möglichen Werten **`AVAILABLE`**, **`AWAY`** und **`DND`** sowie den passenden, sprechenden Labels „Available“, „Away“ und „Do not disturb“!

Schreiben Sie eine Klasse `Account`, die zu jedem Konto den Benutzernamen (`String name`), den aktuellen Status (`State state`) und eine Farbe für dessen Nachrichten (`Color color`) speichert! Dabei soll der Name im Konstruktor übergeben und die Farbe für den Account *zufällig und nicht zu hell (s. Hinweis)* erzeugt werden. Initial ist jeder Account im Zustand `State.AVAILABLE`.

Schreiben Sie eine Klasse **`Message`**, die zu jeder Nachricht deren Inhalt (`text` vom Typ `String`) sowie eine Referenz auf den Absender (`Account`-Instanz) speichert! Der Konstruktor von `Message` soll das Setzen beider Attribute ermöglichen.

Machen Sie die Attribute beider Klassen „privat“ und über Getter/Setter-Methoden verfügbar!

SnatChat – *#GansGeheim!*

Teilaufgabe c) **[10%]**

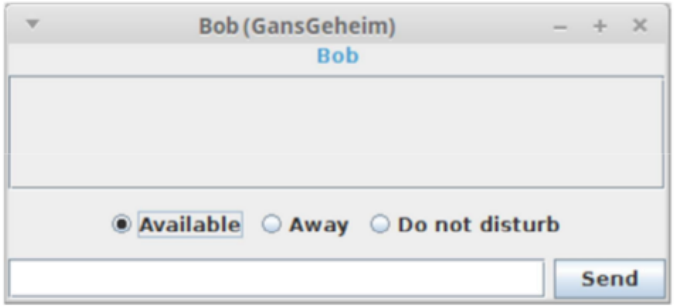
Entwickeln Sie eine Klasse **`SnatChatRoom`** mit (mindestens) folgenden Methoden:

- **`public String getRoomName()`** ermöglicht das Auslesen des Raum-Namens. Der Name des Raums soll in einer Instanz-Variablen (`String roomName`) gespeichert und im Konstruktor von `SnatChatRoom` übergeben werden!
- **`public void register(SnatChatFrontend s)`** ermöglicht die Anmeldung eines Frontends (welches das Interface `SnatChatFrontend` implementiert).
- **`public void unregister(SnatChatFrontend s)`** ermöglicht das Abmelden eines Frontends.
- **`public void sendMessage(Message msg)`** schickt eine Nachricht `msg` an alle angemeldeten Frontends.
- **`public void sendMessage(String text)`** schickt den Nachrichtentext `text` an alle angemeldeten Frontends.

SnatChat – *#GansGeheim!*

Teilaufgabe d) **[10%]**

Schreiben Sie eine Klasse **`SnatChatWindow`**, welche das Interface `SnatChatFrontend` implementiert und eine grafische Benutzeroberfläche in einem eigenen Fenster erzeugt!



In einem Konstruktor soll dem `SnatChatWindow` ein `SnatChatRoom`-Objekt sowie ein `Account` übergeben werden. *(vgl. Beispiel-Code zum Starten / Hinweise)*

Die Benutzeroberfläche eines `SnatChatWindow`s besteht aus der Titelzeile (Format: „*Benutzername (Raumname)*“), dem Benutzernamen in der entsprechenden Farbe des Accounts innerhalb des Fensters, einer `ChatMessagesComponent` (Klasse bereitgestellt auf USB-Stick) für die Anzeige der Nachrichten, Radio-Buttons für die Auswahl des aktuellen Status (der richtige Status ist passend zu der übergebenen Account-Instanz zu selektieren), einem einzeiligen Feld zur Eingabe von Nachrichten („Eingabe“, zunächst leer) und einem Button „Send“ zum Absenden der Nachrichten (zunächst keine Aktion, wird später ergänzt).

Mit Hilfe der bereitgestellten `ChatMessagesComponent` sollen beim Aufruf von…

- … **`receiveMessage(String)`** eine Nachricht mit dem übergebenen Text und der Textfarbe Grau (`java.awt.Color.GRAY`) hinzugefügt werden.
- … **`receiveMessage(Message)`** eine Nachricht in der Form „Absender: Nachrichtentext“ mit der Farbe des Absenders als Textfarbe hinzugefügt werden.

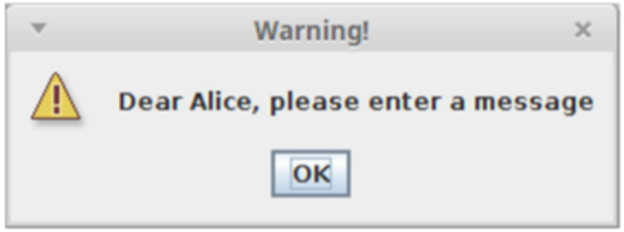
Hinweis: In Teilaufgabe g) wird diese Funktionalität erweitert!

SnatChat – *#GansGeheim!*

Teilaufgabe e) **[10%]**

Erweitern Sie die Klasse `SnatChatWindow` so, dass sie auf Benutzerinteraktionen reagiert:

- Bei der Auswahl eines RadioButtons soll eine einfache Text-Nachricht in der Form „State of user *'username'* is now *'statelabel'*“ verschickt werden (vgl. Abbildung Vorderseite) sowie der Status im `Account`-Objekt aktualisiert werden.
- Beim Drücken des Buttons „Send“ soll…
  - … bei leerem Nachrichtenfeld ein Dialog erscheinen, der den aktuellen Benutzer darauf hinweist, dass eine Nachricht eingeben muss. *(vgl. Abbildung unten)*
  - … bei gefülltem Nachrichtenfeld ein Nachrichten-Objekt mit dem Inhalt des Textfelds als Nachrichtentext und dem `Account` der `SnatChatWindow`-Instanz als Absender generiert und verschickt werden. Im Anschluss soll das Textfeld geleert werden.
- **1 Zusatzpunkt**: Auch das Drücken der Enter-Taste im Textfeld löst das Abschicken aus.



SnatChat – *#GansGeheim!*

Teilaufgabe f) **[10%]**

Erweitern Sie die Klasse `SnatChatRoom` so, dass alle Nachrichten, die über den Raum verschickt werden, auch in eine Textdatei geloggt werden. Hierfür soll jede Nachricht in eine Zeile der Datei „*`raumname`*`.txt`“ geschrieben werden (also „`GansGeheim.txt`“ für den Raumnamen aus der bereitgestellten main-Methode). Besteht die Datei bereits, sollen weitere Nachrichten angehängt werden.

Darüber hinaus soll beim Registrieren einer `SnatChatFrontend`-Instanz die Textdatei eingelesen werden und jede Nachricht als einfache Text-Nachricht direkt an das Frontend geschickt werden.

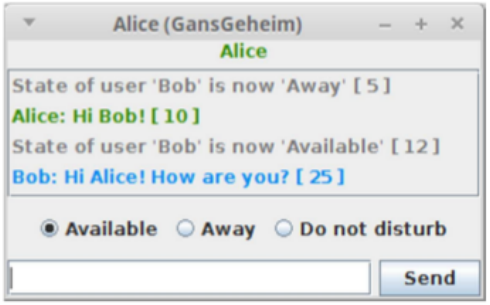
Erweitern Sie die Einlese-Routine für die Log-Datei anschließend so, dass nur maximal die letzten 10 Nachrichten/Zeilen an das Frontend geschickt werden, nicht die komplette Historie.

SnatChat – *#GansGeheim!*

Teilaufgabe g) **[10%]**

Der Nachrichtendienst *SnatChat* legt großen Wert auf Vertraulichkeit. Erweitern Sie die Klasse `SnatChatWindow` daher so, dass eingehende Nachrichten nach 30 Sekunden automatisch aus der Nachrichtenliste entfernt werden.

Für jede Nachricht soll rechts des Nachrichtentextes ein Countdown heruntergezählt werden, wie viele Sekunden die Nachricht noch zu sehen ist.



SnatChat – *#GansGeheim!*

Teilaufgabe h) **[10%]**

Vertraulichkeit ist der Markenkern von *SnatChat* – daher sollen die Log-Dateien nicht im Klartext gespeichert, sondern die Nachrichten verschlüsselt abgelegt werden. Hierfür wurde der Algorithmus „`rot13`“ ausgewählt. Implementieren Sie hierfür in der Klasse `Message` eine statische Methode

```
public static String rot13(String message)
```

welche die Zeichenkette `message` durchläuft und auf jedes Zeichen folgende Regeln anwendet und das Ergebnis zurückgibt:

- Ersetzt werden nur Zeichen von `A-Z` und `a-z`, alle anderen Zeichen bleiben gleich
- Alle Zeichen von `A-M` und `a-m` werden durch das Zeichen 13 Buchstaben *später* im Alphabet ersetzt
- Alle Zeichen von `N-Z` und `n-z` wird das Zeichen 13 Buchstaben *früher* im Alphabet ersetzt

Sorgen Sie dafür, dass vor dem Schreiben der Zeile in die Datei die „Verschlüsselung“ auf die zu schreibende Zeichenkette angewandt wird!

Der große Vorteil von `rot13` ist, dass durch doppeltes Anwenden alles wieder lesbar wird. Sorgen Sie dafür, dass beim Lesen aus der Log-Datei auf jede eingelesene Zeile wiederum der rot13-Algorithmus angewandt wird um die Zeilen wieder lesbar zu machen.

SnatChat – *#GansGeheim!*

Hinweise:

Starten:

Starten Sie die Anwendung mit einer Klasse `SnatChat` (s. USB-Stick), etwa

```
public class SnatChat {
   public static void main(String[] args) {
      SnatChatRoom room = new SnatChatRoom("GansGeheim");
      room.register( new SnatChatWindow(room, new Account("Bob") ) );
      room.register( new SnatChatWindow(room, new Account("Alice") ) );
   }
}
```

Erzeugen einer zufälligen, nicht zu hellen Farbe:

Nutzen Sie hierfür `new java.awt.Color(int r, int g, int b)`. Für `r`, `g` und `b` sind jeweils **zufällige Werte von 0 bis 200 (beide inklusive)** zu erzeugen.

Verwendung der bereitgestellten Klasse `ChatMessagesComponent`:

Anstatt eines `JPanels` für das Hinzufügen der Nachrichten können Sie die Klasse `ChatMessagesComponent` verwenden, welche sich um das Layouten (inkl. Scroll-Balken) der Nachrichten kümmert. Die Verwendung kann beispielhaft so aussehen:

```
ChatMessagesComponent chatMessages = new ChatMessagesComponent();
JLabel myLabel = new JLabel("A Message for Bob");  // Label mit neuer Nachricht

chatMessages.add(myLabel); // Hinzufügen einer Nachricht
chatMessages.remove(myLabel); // Entfernen einer Nachricht

frame.add(chatMessages);
```